
LECTURE NOTES

Exam preparations

AI505 - Optimization

Simon Holm
June, 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

Bracketing	5
Fibonacci Search	5
Golden Ratio Search	5
Quadratic fit search	5
Lipschitz continuous	5
Bisection method	5
A simple minimization problem	6
Taylor expansion	7
Convexity	7
Directional derivative	8
Positive Definiteness	8
Descent Direction Iteration	8
Line Search	8
Backtracking	8
Wolfe conditions	9
Strong wolfe	9
Strong Backtracking	9
Conjugate Gradient	9
Rate of convergence	10
Q-linear (quotient-linear)	10
Q-superlinear (quotient-superlinear)	10
Q-quadratic (quotient-superlinear)	10
R-linear (root-linear)	11
Trust Region Methods	11
Termination Conditions	11
Steepest Descent	11
Conjugate gradient method	11
Momentum	11
Newton method	12
Convergence	12
Secant method	12
Quasi-Newton methods	12
Derivative free methods	12
Stable Step Size	13
Noisy Descent	13
Stochastic Gradient Descent	13
Mesh Adaptive Direct Search	13
Simulated Annealing	13
Metropolis acceptance criterion:	13
Cross entropy/Max likelihood	13
Natural Evolution Strategies	14
CMA-ES	14

COCO benchmarking	14
Population-based methods	15
Differential Evolution	15
Swarm	15
Sampling Plans	16
Full Factorial Design	16
Random Sampling	16
Uniform Projection Plans	16
Stratified Sampling	16
Space Filling Metrics	16
Space-Filling Subsets	16
Quasi-Random Sequences	16
Machine Learning as Optimization	17
Expected risk vs empirical risk	17
Stochastic approach	17
Batch approach	17
Noise Reduction	17
Second order methods	18
SG convergence analysis	18
Constrained Optimization	18
Transformations to Remove Constraints	18
Lagrangian Function	18
KKT (Karush-Kuhn-Tucker conditions)	18
Primal form	19
Dual form	19
Penalty methods	19
Barrier functions	19
LP	19
Model form nonlinear to LP	19
Simplex problems	20
Inequality constraints	20
Equality constraints	20
Conversion	20
The simplex algorithm	20
Dual certification	21
Discrete Optimization	21
Exact methods	21
Heuristic Methods	21
ILP	22
LP rounding	22
Cutting Plane method	22
CG method	22
Branch and Bound	22

Dynamic programming	23
Constraint programming	23

Bracketing

For $a < b < c$

there exists a minimum in interval $[a, c]$ if

$$f(a) < f(b) \wedge f(c) < f(b)$$

Fibonacci Search

When we know how many evaluations we have available

Places evaluations such that one evaluation always carries over to the next iteration.

Golden Ratio Search

If one were to use fibonacci with ∞ function evaluations. it would converge to a avg distance of ϕ

Golden Ratio just uses ϕ

Quadratic fit search

Iteratively fits a quadratic and uses that minimum

Lipschitz continuous

Upper bound on the derivative

For some constant L ,

$$|\nabla f| \leq L$$

Bisection method

Find roots since if $\text{sign}(\nabla f(a)) \neq \text{sign}(\nabla f(b))$ then $[a, b]$ guaranteed to contain a root.

So cut the midpoint and choose the side where this property still holds

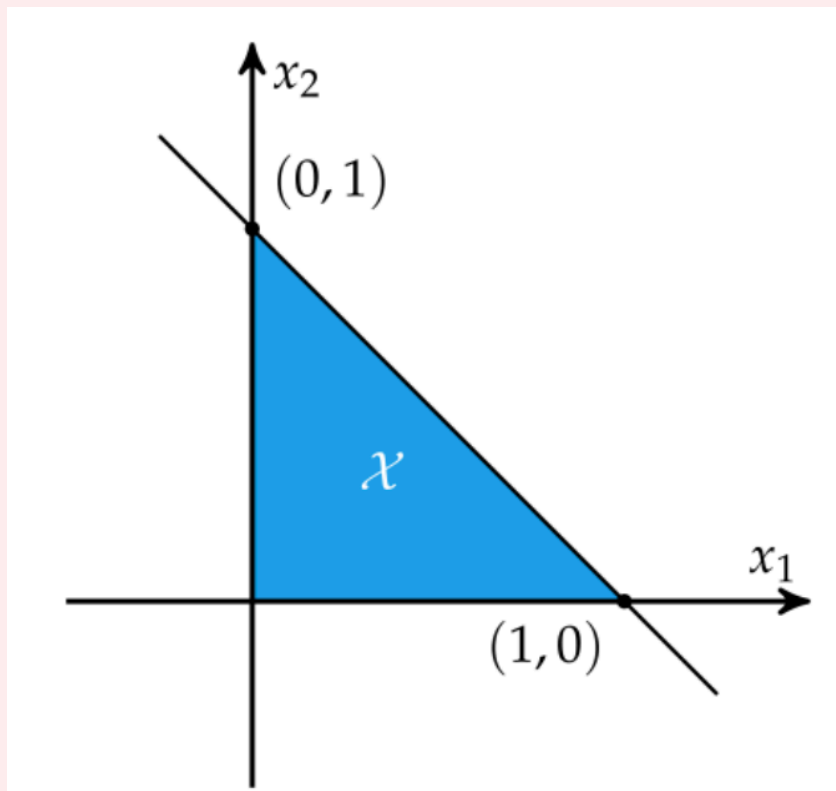
A simple minimization problem

Defining a minimization problem such that x lies within a feasible set of solutions

$$\begin{aligned} \min_x & f(x) \\ \text{s.t. } & x \in \mathcal{X} \end{aligned}$$

Example

$$\begin{aligned} \min_{x_1, x_2} & f(x_1, x_2) \\ \text{s.t. } & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_1 + x_2 \leq 1 \end{aligned}$$



Taylor expansion

Taylor expansion

Let a be a fixed point. The Taylor expansion approximates $f(x)$ near a using only information about f at a .

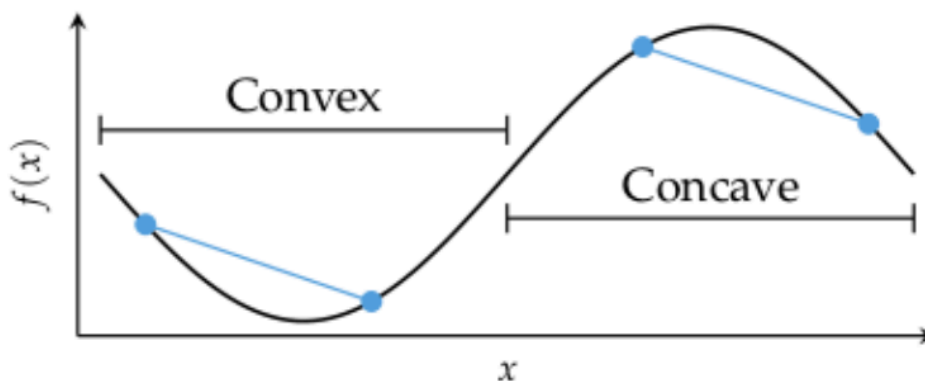
$$f(x) = f(a) + \nabla f(a)^T (x - a) + \frac{1}{2} (x - a)^T \nabla^2 f(a) (x - a)$$

$$f(a + h) = f(a) + \frac{\nabla f(a)}{1!} h + \frac{\nabla^2 f(a)}{2!} h^2 + O(\|h\|^3)$$

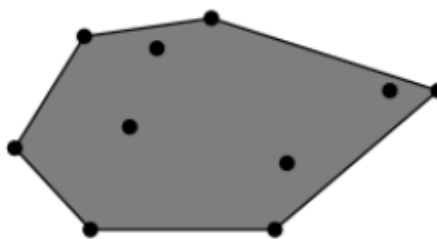
Convexity

$$f : \mathbb{R}^n \rightarrow \mathbb{R}$$

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y)$$



For a convex hull:



the convex hull of X

$$\text{conv}(X) = \left\{ \sum_i \lambda_i x_i \mid x_i \in X, \lambda_i \geq 0, \sum_i \lambda_i = 1 \right\}$$

Directional derivative

$$\nabla_s f(x) = \nabla f(x)^T s = \frac{\partial f(x)}{\partial x_1} s_1 + \frac{\partial f(x)}{\partial x_2} s_2 + \dots + \frac{\partial f(x)}{\partial x_n} s_n$$

Positive Definiteness

Positive definiteness

A square, symmetric matrix A is **positive definite** if $x^T A x$ is positive for all points other than the origin

$$A \succ 0 = x^T A x > 0 \quad \forall x \in \mathbb{R} \setminus \{0\}$$

Positive semidefiniteness

A square, symmetric matrix A is **positive semidefinite** if $x^T A x$ is always nonnegative

$$A \succeq 0 = x^T A x \geq 0 \quad \forall x \in \mathbb{R}$$

Descent Direction Iteration

$$x_{k+1} = x_k + \alpha_k d_k \quad \text{where } d = -B_k^{-1} \nabla f(x_k)$$

Note that this is because in Newton $B_k = \nabla^2 f(x_k)$

And for 1st order methods like GD $B_k = I$

Line Search

$$\alpha^* = \min_{\alpha \geq 0} f(x_k + \alpha d_k)$$

Backtracking

$$\alpha_0 = 1$$

Then decrease alpha such that

$$f_{k+1} \leq f_k + \beta \alpha \nabla_{d_k} f_k \quad \text{where } \beta \in [0, 1]$$

Wolfe conditions

1. First Wolfe Condition: Sufficient Decrease (Armijo condition)

$$f(x_{k+1}) \leq f(x_k) + \beta \alpha \nabla f(x_k)^T \nabla_{d_k} f(x_k)$$

1. Second Wolfe Condition: Curvature Condition

$$\nabla_{d_k} f(x_{k+1}) \geq \sigma \nabla_{d_k} f(x_k)$$

Strong wolfe

Modify Curvature Condition such that

$$|\nabla_{d_k} f(x_{k+1})| \geq \sigma |\nabla_{d_k} f(x_k)|$$

Strong Backtracking

- Bracketing-phase: golden ratio
- Zoom phase: bisection or some interpolation (like quadratic fit)

Conjugate Gradient

Using line search results in

$$\nabla f(x_{k+1})^T \nabla f(x_k) = 0 \quad (\nabla f(x_{k+1}) \perp \nabla f(x_k))$$

$$d_i^T H d_j = 0, \quad \forall i \neq j \text{ Vectors are not orthogonal.}$$

Where $H \succeq 0$

Generally $d_0 = -\nabla f(x_0)$ and,

$$d_{k+1} = -\nabla f(x_{k+1}) + \beta_k \nabla f(x_k)$$

Where

$$\beta_k^{\text{Fletcher-Reaves}} = \frac{\nabla f(x_{k+1})^T \nabla f(x_{k+1})}{\nabla f(x_k)^T \nabla f(x_k)}$$
$$\beta_k^{\text{Polak-Ribière}} = \frac{\nabla f(x_{k+1})^T (\nabla f(x_{k+1}) - \nabla f(x_k))}{\nabla f(x_k)^T \nabla f(x_k)}$$

Rate of convergence

We need to make sure of method convergence.

by

Theorem: Zoutendijk condition

Consider any iteration of the form $x_{k+1} \leftarrow x_k + \alpha_k d_k$, where d_k is a descent direction and α_k satisfies the Wolfe conditions. Suppose that f is bounded below in $\mathbb{R}^n$ and that f is continuously differentiable in an open set N containing the level set $L = \{x : f(x) \leq f(x_0)\}$, where x_0 is the starting point of the iteration. Assume also that the gradient ∇f is Lipschitz continuous on N , that is, there exists a constant $L > 0$ such that:

$$|f(x) - f(y)| \leq L|x - y|, \forall x, y \in N$$

Then:

$$\sum_{k \geq 0} \cos^2 \theta_k \|\nabla f_k\|^2 < \infty$$

This means that $\cos^2 \theta_k \|\nabla f_k\|^2 \rightarrow 0$

Since $\cos \theta_k$ can be bounded by a constant $\frac{1}{M}$ we know that

$$\|\nabla f_k\|^2 \rightarrow 0$$

For a sequence of x in $\mathbb{R}^n$

$$\text{Q-linear} \approx \text{R-linear} < \text{Q-superlinear} < \text{Q-quadratic}$$

Q-linear (quotient-linear)

There exists $r \in (0, 1)$ such that

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \leq r \quad \forall k \gg 1$$

Q-superlinear (quotient-superlinear)

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} = 0$$

Q-quadratic (quotient-superlinear)

For some constant $M > 0$

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^2} \leq M \quad \forall k \gg 1$$

R-linear (root-linear)

For a sequence of nonnegative scalars $\{v_k\}$ where $v_k \rightarrow 0$

$$\|x_{k+1} - x^*\| \leq v_k \quad \forall k \gg 1$$

Trust Region Methods

$$\begin{aligned} & \min_{x'} \hat{f}(x') \\ & s.t. \quad \|x - x'\| \leq \delta \end{aligned}$$

δ can be expanded or contracted by η

$$\eta = \frac{\text{real improvement}}{\text{predicted improvement}} = \frac{f(x) - f(x')}{f(x) - \hat{f}(x')}$$

Termination Conditions

- Maximum Iterations: $k > k_{\max}$
- Absolute Improvement: $f(x_k) - f(x_{k+1}) < \varepsilon_a$
- Relative Improvement: $f(x_k) - f(x_{k+1}) < \varepsilon_r |f(x_k)|$
- Gradient Magnitude: $\|\nabla f(x_{k+1})\| < \varepsilon_g$

Steepest Descent

$$d_k = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}$$

Conjugate gradient method

In generating its set of conjugate vectors, it can compute a new vector d_k by using only the previous vector d_{k-1} .

$$d_k = -\nabla f(x_k) + \beta_k d_{k-1}$$

Where

$$\beta_k = \frac{\nabla f(x_k)^T A d_{k-1}}{d_{k-1}^T A d_{k-1}}$$

Momentum

$$\begin{aligned} v_{k+1} &= \beta v_k - \alpha \nabla f(x_k) \\ x_{k+1} &= x_k + v_{k+1} \end{aligned}$$

Newton method

Multivariate update rule

$$x_{k+1} = x_k - H_k^{-1} \nabla f(x_k)$$

Convergence

When $\nabla^2 f(x^*) \succ 0$ and $\nabla f(x^*) = 0$ (locally bowl shaped) $\implies \min_x = x^*$

When f is strong convex, then $\nabla^2 f(x) \succ 0, \quad \forall x$

Secant method

Approximate $f''(x_k) \approx \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}}$ So

$$x_{k+1} = x_k - \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}} f'(x_k)$$

Quasi-Newton methods

Approximate $Q_k \approx H^{-1}$ iteratively

Derivative free methods

- Cyclic Coordinate Search
 - line search in alternating coordinate directions
- Powell's method
 - Drops already seen directions
- Hooke-Jeeves
 - evaluate $f(x)$ and $f(x \pm \alpha e_i)$ and select best one
 - If no improvements are found shrink α
- Generalized pattern search
 - Hooke-Jeeves but with $n + 1$ directions
- Nelder-Mead
 - Reflection, expansion, contraction, shrinkage.
- DIRECT - Divided rectangles
 - Balances interval size and function evaluation

Stable Step Size

Consider

$$f(x) = \frac{1}{2} x^T Q x - b^T x, \quad \text{where } Q = Q^T \succ 0$$

Then $\nabla f(x) = Qx - b$ One can derive that the step size that stabilizes the iteration

$$0 < \alpha < \frac{2}{\lambda_{\max}(Q)}$$

Noisy Descent

$$x_{k+1} = x_k + \alpha \nabla f(x_k) + \varepsilon_k \quad \text{where } \varepsilon_k \sim \mathcal{N}(0, \sigma_k^2)$$

Stochastic Gradient Descent

$$\nabla f(x) \approx \frac{1}{|B|} \sum_{i \in B} \nabla f_i(x)$$

Mesh Adaptive Direct Search

Similar to generalized pattern search but uses controlled randomness to generate directions

Simulated Annealing

$$x' = x + \varepsilon, \quad \text{where } \varepsilon \in \mathcal{N}(0, T)$$

Metropolis acceptance criterion:

Let $\Delta = f(x') - f(x)$ Then

$$p(x, x') = \begin{cases} 1 & \text{if } \Delta \leq 0 \\ e^{-\Delta/t_k} & \text{if } \Delta > 0 \end{cases}$$

Cross entropy/Max likelihood

Fit a distribution to elite points (lowest function evaluation)

$$\theta^* = \arg \min_{\theta} - \sum_{i=1}^N \log p(x_i | \theta) = \arg \min_{\theta} \prod_{i=1}^N p(x_i | \theta)$$

One can model data and use to sample from and/or make predictions.

Natural Evolution Strategies

Now gradient descent:

$$\theta_{k+1} = \theta_k - \alpha \frac{1}{N} \sum_{i=1}^N f(x_i) \nabla_{\theta} \log p(x_i | \theta)$$

Where $p(\cdot | \theta) = \mathcal{N}(\mu, \sigma^2)$

Similar to CE but all points.

CMA-ES

Same but with covariance so that it can utilize multivariate data.

$$x \sim \mathcal{N}(\mu, \sigma^2 \Sigma)$$

mean is weighed average of m -elites

CMA-ES uses cumulative vectors p_1 (length) and p_2 (shape), which a decaying memory vector $\in \mathbb{R}^n$ (kinda like rnn's)

COCO benchmarking

Benchmark your method against 24 functions

- n dimensions
- i function (1-24)
- j instance (rotate/shift)

Population-based methods

Optimize a collection of **individuals**

We only keep track of chromosomes (each individual is interpreted as a set of chromosome, commonly vectors $\in \mathbb{R}^n$)

- **Selection**
 - Determine which individuals pass their genetic information on to the next generation
 - **Truncation** selection: truncate the lowest performers
 - **Tournament** selection: selects fittest out of k randomly chosen individuals
 - **Roulette Wheel** selection: individuals are chosen with probability proportional to their fitness
- **Crossover**
 - Combine chromosomes from two parents to create child
 - **Single-point**: swap occur after a single point
 - **Multiple-point**: swap z times
 - **Uniform** crossover: Each chromosome has 50% chance of crossover
- **Mutation**: randomly mutate some individuals to support explorations.
 - Mutate by *mutation rate*
- **Replace**: Choose the new populations using all above
 - **Goal**: find the best population, but avoid duplicates.

Differential Evolution

Choose 3 random distinct individuals a, b and c

Then

$$z = a + w(b - c)$$

$$x'_i \begin{cases} z_i & \text{if } i = j \text{ where } j \sim \mathcal{U}\{1, \dots, n\} \\ x_i & \text{otherwise} \end{cases}$$

Swarm

Now let each individual track,

- position,
- velocity
- Own best position
- Global best position

As particles fly toward the current global best, they pass through different regions of the search space. If one of them stumbles across a better point along the way — that becomes the new global best, and the whole swarm redirects toward it.

Sampling Plans

Choosing how to initialize points before optimization begins. There are several methods.

- try optimizing multiple starting points and keep best result
- if f is expensive, one can sample points from a *surrogate model*

Surrogate model: cheap fake version of f

Full Factorial Design

Sample points uniformly and evenly spaced samples across domain Simple, easy to design, but grows with n^m and can miss local features

Random Sampling

Sample randomly from some distribution over $[a_1, b_1] \times \dots \times [a_n, b_n]$

Uniform Projection Plans

Sample m points from $m \times m$ with the premise that every dimension must be uniformly covered (like N-queens but with rooks).

Stratified Sampling

Fix problem with local accuracy that full factorial has. Place points uniformly random within their square

Space Filling Metrics

Fill the space so everything is covered. (measure how spread points are)

- Like pairwise distance

Space-Filling Subsets

Choose a subset of the space what best represents the whole space.

Quasi-Random Sequences

Optimize the points such that

$$\int f(x) \, dx \approx \frac{1}{M} \sum_{i=1}^M f(x_i)$$

Machine Learning as Optimization

Predict

$$h(x; w), \quad \text{given } \{(x_i, y_i)\}_{i=1}^n$$

.

Expected risk vs empirical risk

Ideally, you want to minimize the expected risk

$$R(w) = \mathbb{E}[f(w; \xi)] = \int \ell(h(x; w), y) \, dP(x, y)$$

Since $P(x, y)$ is unknown, we minimize **empirical risk** instead.

$$R_n(w) = \frac{1}{N} \sum_{i=1}^n \ell(h(x_i; w), y_i)$$

Usually regularize $R_{n(h)} + \lambda \Omega(h)$

Stochastic approach

Stochastic Gradient like

$$w_{k+1} = w_k - \alpha_k \nabla f_{i_k}(w_k)$$

- very cheap, though might not always be optimal.

Batch approach

use the full gradient of a batch instead of the whole dataset.

$$w_{k+1} = w_k - \alpha_k \nabla R_n(w_k) = w_k - \frac{\alpha_K}{N} \sum_{i=1}^N \nabla f_i(w_k)$$

Noise Reduction

Stochastic gradient methods can have noisy gradients leading to variance (and overfitting).

- Dynamic Sampling:
 - increase the mini-batch size used in the gradient computation
- Gradient aggregation
 - aggregate new gradients with the previous iterations
 - SAGA - Stochastic average gradient algorithm
 - if the average of gradient went in direction d it should also infer the new gradient in direction d

Second order methods

- Diagonal scaling
 - Scale only using a diagonal hessian

$$H_k \approx \text{diag}(H_k)$$

- It still holds curvature information, just not as much.
- quasi-Newton
 - construct approximations to the Hessian using only gradient information (L-BFGS)
- Gradient free
 - Use system of linear equations to find s_k from $H_k s_k = -\nabla f$

SG convergence analysis

- Strongly convex + fixed α = never converge. You need to shrink α (slower).
- Non convex cannot be bound since there might be many local minima. you might approach a stationary point though. When shrinking α you get closer to a minima, but slower.

Constrained Optimization

1. Equality constraints: $h(x) = 0$
2. Inequality constraints: $g(x) \leq 0$

Transformations to Remove Constraints

$$\min_{x \in [a, b]} f(x)$$

Instead define $\mathcal{T}_{a,b} : \mathbb{R} \rightarrow [a, b]$

$$\min_{\hat{x} \in \mathbb{R}} f(\mathcal{T}(\hat{x}))$$

Lagrangian Function

$$\mathcal{L}(x, \mu, \lambda) = f(x) + \sum_i \mu_i g_i(x) + \sum_j \lambda_j h_j(x)$$

KKTC (Karush-Kuhn-Tucker conditions)

For any x^*

1. Primal feasibility: $g(x^*) \leq 0, h(x^*) = 0$
2. Dual feasibility: $\mu \geq 0$
3. Complementary slackness: $\mu \cdot g(x^*) = 0$
4. Stationarity: $\nabla f(x^*) + \mu \cdot \nabla g(x^*) + \lambda \cdot \nabla h(x^*) = 0$

Primal form

$$\min_x \max_{\mu \geq 0, \lambda} \mathcal{L}(x, \mu, \lambda)$$

Dual form

$$\max_{\mu \geq 0, \lambda} \min_x \mathcal{L}(x, \mu, \lambda)$$

Penalty methods

Examples:

- Count penalty

$$p_{\text{count}}(x) = \sum_i (g_i(x) > 0) + \sum_j (h_j(x) \neq 0)$$

- Quadratic penalty

$$p_{\text{quadratic}}(x) = \sum_i \max(g_i(x), 0)^2 + \sum_j h_j(x)^2$$

- Mixed Penalty

$$p_{\text{mixed}}(x) = p_{\text{count}} + p_{\text{quadratic}}$$

Barrier functions

$$p_{\text{barrier}}(x) = - \sum_i \log(-g_i(x))$$

LP

when both the objective and all constraints are linear.

Model form nonlinear to LP

$$\min_x \|Ax - b\|_1$$

$$\min_x 1^T s$$

$$s.t. \quad Ax - b \leq s$$

$$-(Ax - b) \leq s$$

Simplex problems

Each inequality constraints are half-spaces, where equality would reduce the dimensions.

multiple solutions types

Inequality constraints

standard way of thinking using $\leq$

minimize problem such that something is less than

Equality constraints

Easier to solve using *s.l.e.* to solve for variables x_i

Conversion

Introduce *slacks*

$$a_i^T x \leq b \implies a_i^T x + s_i = b$$

The simplex algorithm

$$\mathcal{L}(x, \mu \geq 0, \lambda) = c^T x - \mu^T x - \lambda(Ax - b)$$

$$\nabla \mathcal{L} = c - \mu - \lambda^T A = 0 \implies A^T \lambda + \mu = c$$

Split into (B, N)

$$\begin{cases} A_B^T \lambda + \mu_B = c_B \\ A_N^T \lambda + \mu_N = c_N \end{cases} \implies \lambda = (A_B^T)^{-1} c_B \quad \text{since } x \geq 0, \mu_B = 0$$

Now

$$A_N^T (A_B^T)^{-1} c_B + \mu_N = c_N \iff \mu_N = c_N - A_N^T (A_B^T)^{-1} c_B$$

One must maintain

$$A_B x'_B + A_{\{q\}} x'_{\{q\}} = A_B x_B = Ax = B$$

This is done by updating $x_B \rightarrow x'_B$ by

$$x'_B = x_B - A_B^{-1} A_{\{q\}} x'_{\{q\}}$$

Then the objective value function is updated $c^T x' = c^T x + \mu_{\{q\}} x'_{\{q\}}$ We can initialize the problem by populating x with values that fit into $Ax = b$

- **Pivoting:** move elements from/to B and N
 - μ_N defines a cost of “moves” when partition elements in B and N
 - Choose the leaving candidate the yields the smallest x' (**minimum ratio test**)
 - for multiple entering candidates with $\mu_{\{q\}} < 0$ several
 - If all components of μ_N are non-negative, we have found a global optimum.

Dual certification

Verify that a solution is optimal given that for primal solution p^* and dual solution d^*

Weak duality guarantees that the duality gap:

$$d^* - p^* \geq 0$$

If $d^* = p^*$ then p^* must be the unique optimal value. That is for $f(x) = p^* \implies x^*$

Duals might be infeasible. **Example:** if $p^* = -\infty$ then $d^* \not\leq -\infty$.

Discrete Optimization

Space is discrete. Variables can only take specific, distinct values. These problems tend to be computationally expensive, so one can utilize heuristics.

Exact methods

- ILP
- Combinatorial optimization algorithms
- SAT
- Dynamic
- Constraint

Heuristic Methods

- Greedy
- Local search
- Metaheuristics
 - Genetic
 - SA

ILP

Variables can be $\mathbb{Z}$, $\mathbb{N}_0$ or $\mathbb{B}$

- Mixed ILP (not all variables **has** to be integers)

LP rounding

Solve LP-problem and round the solution to integer. Typically round to suboptimal point so we don't round outside the feasible space. This can lead to problems where you essentially need to start searching again if the optimal integer is far away from the x^*

Cutting Plane method

Assume that LP $f(x) = x^*$ where x^* is fractional. when solving x^* is not a valid integer solution. Find cutting plane (linear constraints) such that the problem is constrained to exclude the infeasible non-integer space beyond the optimal integer x^*

$$\begin{aligned} \max_x & c^T x \\ \text{s.t.} & Ax \leq b \\ & x \in \mathbb{Z}^n \end{aligned}$$

CG method

Take the partition that

$$A_B x_B^* + A_N x_N^* = b \implies x_B^* = A_B^{-1} b - A_B^{-1} A_N x_N^*$$

Now floor some of the constraints such that

$$A_B b - \lfloor A_B b \rfloor - \underbrace{(A_B^{-1} A_N - \lfloor A_B^{-1} A_N \rfloor)}_{=0} x_N^* \leq 0$$

$$A_B b - \lfloor A_B b \rfloor \leq 0 \quad (\text{integer})$$

Branch and Bound

$$\min c^T x \quad \text{where } x \in S$$

$$S = S_1 \cup S_2 \cup \dots \cup S_k$$

Define upper bound and lowerbound

$$\underline{z}^k \leq z^k \leq \bar{z}^k$$

At each node one can prune any branch where $\underline{z}^k \geq \bar{z}^k$

So the algorithm naturally stops when all expanded nodes have been pruned or **solved**

Dynamic programming

optimal substructures + overlapping subproblems

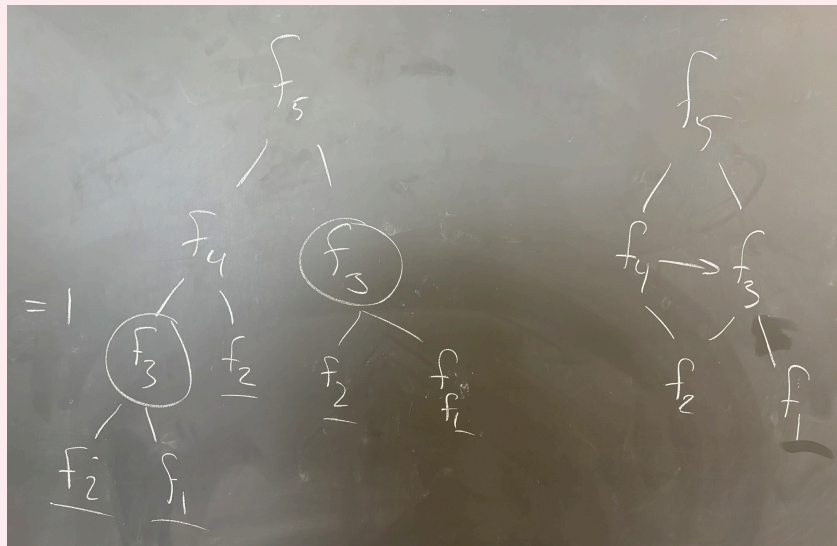
recursion can easily benefit from this, since it includes overlapping quite a bit

Example: Fibonacci

Let

$$F_n = F_{n-1} + F_{n-2} \quad \text{where } F_1, F_2 = 1$$

Then we can optimize storing known F's



For TSP instead of brute force one can model so that

$$z^* = g(i, S) = \min_{k \in S} \{c_{i,k} + g(k, S - \{k\})\}$$

Constraint programming

Assign values to variables such that some constraints are satisfied.

Goal: feasibility

- variables x
- Domain $x \in \mathcal{D}$
- Constraints: some rules (eg. adjacency)

The **core** of CP 😊:

- Modelling
- Inference: after assigning a variable, propagate the consequence to reduce domains

- Heuristics: Guide when multiple options are valid. (like assign most constrained first)
- Symmetry: some assignments might lead to a symmetric result. compensate for this.
-